Machine-Checked Linear Temporal Logic for the UEFI Capsule Update Process: A Foundational Proof of Authenticity, Anti-Rollback, and Responsiveness

Vincent Zimmer
Anduril Industries
vzimmer@anduril.com

***Abstract*—Platform firmware update is among the highest-value targets in the modern trust stack: a malicious capsule survives OS reinstallation and persists below every software defense. UEFI standardizes this path via `UpdateCapsule` and the Firmware Management Protocol (FMP), but the security argument exists only in prose and reference C code. We present the first machine-checked temporal-logic treatment of the UEFI capsule update process. We construct a compact labelled-transition model of `UpdateCapsule`, mandatory reset, post-reset coalescing, FMP authentication, and image application, with an explicit lowest-supported-version floor and a staged digest whose signature verdict is computed at authentication time. We embed Linear Temporal Logic (LTL) shallowly over its runs in Lean 4. We prove authenticity; local and global monotonicity of the stored lowest-supported-version floor; an apply guard showing that every applied image was signature-valid and at or above that floor; reset ordering; and, under the forced-good-path assumption of the interruption-free model, responsiveness: a staged capsule whose digest authenticates and whose version is at or above the floor eventually reaches `Applied`. All theorems are kernel-checked; the axiom audit confirms reliance only on `propext` and `Quot.sound` with zero `sorry`. We map the model onto EDK II `FmpDevicePkg`, where the proofs surface latent temporal and fidelity questions around pre-state guards, lowest-supported-version checking, and external interruption. Finally, we carry guarantees end to end: the model is cross-validated by TLC, reproved unbounded via TLAPS, and the good-path logic is reimplemented as a safe-Rust decision-logic prototype, extracted to Lean by Aeneas, and related to the proved `Step` relation.**

***Index Terms*—UEFI, firmware security, capsule update, anti-rollback, linear temporal logic, formal verification, Lean 4, interactive theorem proving.**

Accepted for publication at the 2026 IEEE Cyber Awareness and Research Symposium (CARS). https://ieee-cars.org/

© 2026 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

I. Introduction

Firmware sits at the root of the platform trust chain. Code that executes before the operating system—the boot ROM, the early silicon initialization, and the UEFI firmware volume—establishes the measurements and the authentication decisions on which all later defenses depend. An attacker who can write the SPI flash that holds this code obtains a foothold that survives OS reinstallation and is invisible to OS-resident detection. The threat is not hypothetical: *Thunderstrike* [1], *LoJax* (the first in-the-wild UEFI rootkit) [2], and *BlackLotus* [3] all targeted the firmware layer, and BootHole [4] and PixieFail [5] demonstrate that the UEFI reference implementation itself is a recurring source of exploitable defects. NIST codifies the defensive requirements—authenticated update and anti-rollback chief among them—in SP 800-147B [6] and SP 800-193 [7].

The mechanism by which firmware is *updated* is therefore a load-bearing component of platform security. UEFI standardizes it: the `UpdateCapsule` runtime service [8] hands a signed *capsule* to the firmware, which (typically after a reset) authenticates and applies it through the Firmware Management Protocol (FMP) [9]. The de-facto reference implementation, EDK II [10], realizes this in `FmpDevicePkg`, including the lowest-supported-version logic that enforces anti-rollback.

Despite its criticality, the *argument* that this process is secure exists only as prose in the specification and as C code in the reference implementation. There is no machine-checked statement of what the update flow guarantees, let alone a proof. This is precisely the kind of small, sharp, security-defining state machine for which interactive theorem proving has proven decisive in adjacent domains: seL4 [11], CompCert [12], Ironclad [13], and Komodo [14].

A. Contributions

- A compact, auditable labelled-transition model of the UEFI capsule update process (Section III), faithful to

the `UpdateCapsule`→reset→FMP→apply flow, with an explicit lowest-supported-version floor and an abstract authentication oracle over the staged capsule.
- A shallow embedding of LTL over the model's infinite runs in Lean 4 (Section IV), with the standard $\mathbf{G}, \mathbf{F}, \mathbf{X}$ modalities and a run-validity judgment.
- Machine-checked proofs of five temporal property families (Section V)—authenticity, floor-based local and global anti-rollback, the apply-guard, reset-ordering, and qualified responsiveness on the forced good path—together with a kernel-level axiom audit (Section VI).
- A mapping back to EDK II and a discussion of the fidelity and temporal issues the proof effort surfaced (Section VII): shipped code checks the stored lowest-supported-version floor, and apply guards are necessarily pre-state properties.
- An *end-to-end* extension (Section VIII) that carries the same guarantees down to executable code: the model is cross-checked by TLC and proved unbounded in TLAPS, and the good-path decision logic is reimplemented in safe Rust and translated to Lean by Aeneas. A refinement bridge relates that prototype to the proved `Step` relation; this is not a verification of EDK II, its cryptographic libraries, or its flash-writing code.

The complete development is ~300 lines of Lean and compiles in seconds with no external library dependency.

II. Background

A. The UEFI capsule update flow

A *capsule* is a contiguous blob, prefixed by an `EFI_CAPSULE_HEADER`, that the OS passes to firmware via the `UpdateCapsule()` runtime service [8, §8.5.3]. For a firmware *payload* capsule the typical sequence is:

(1) The OS stages the capsule in memory and calls `UpdateCapsule` with the `CAPSULE_FLAGS_PERSIST_ACROSS_RESET` (and usually `INITIATE_RESET`) flag set.
(2) The platform performs a *reset*. The capsule region is preserved across the reset (memory persistence or a reserved store).
(3) On the next boot, firmware *coalesces* the scattered capsule blocks and locates the FMP instance for the target device.
(4) FMP `CheckImage` authenticates the payload: a digital-signature check *and* a version check against the device's lowest supported version (anti-rollback) [9].
(5) If both checks pass, FMP `SetImage` writes the payload and the running firmware version advances; otherwise the capsule is rejected.

Two security obligations dominate. **Authenticity**: only a correctly-signed image is ever written. **Anti-rollback**: an attacker must not be able to install an older (signed but vulnerable) image to re-introduce a patched defect—a downgrade attack. NIST SP 800-193 makes both mandatory for "protection" and "resiliency" claims [7].

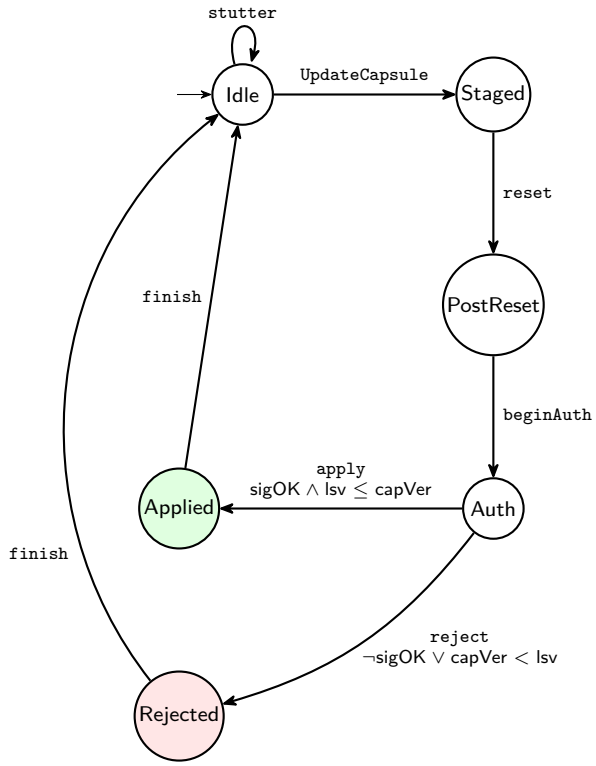


Fig. 1. The capsule-update phase automaton. Green is the success sink (Applied), red the failure sink (Rejected); both return to Idle. The `apply` edge is guarded by signature validity and the stored lowest-supported-version floor; its negation guards `reject`, making the two edges jointly exhaustive from Auth.

B. Linear Temporal Logic

LTL [15], [16], [17] reasons about properties of infinite sequences (*runs*) of system states. Over a set of atomic state predicates, formulas are built with Boolean connectives and the temporal modalities $\mathbf{X}$ (next), $\mathbf{G}$ (globally), $\mathbf{F}$ (eventually), and $\mathbf{U}$ (until). A run $\sigma = s_0 s_1 s_2 \cdots$ satisfies $\mathbf{G}\ \varphi$ iff φ holds at every position, and $\mathbf{F}\ \varphi$ iff φ holds at some position. Safety properties ("nothing bad ever happens") correspond to $\mathbf{G}$ over a predicate; liveness ("something good eventually happens") to $\mathbf{F}$ [18], [19]. We adopt the standard semantics and embed it *shallowly*: an LTL formula is simply a Lean predicate on a run and a position, so the LTL operators are ordinary higher-order functions and proofs are ordinary Lean terms.

III. System Model

We model the update process as a labelled transition system over a finite-field state record. The phase automaton is shown in Fig. 1.

Definition 1 (State)**.** A state $s \in \mathcal{S}$ is a record

$$s = \langle\, \mathsf{phase},\ \mathsf{fwVer},\ \mathsf{lsv},\ \mathsf{capPresent},\ \mathsf{capVer},\ \mathsf{digest},\ \mathsf{sigOK},\ \mathsf{reset} \,\rangle$$

where phase ranges over $\{\mathsf{Idle}, \mathsf{Staged}, \mathsf{PostReset}, \mathsf{Auth}, \mathsf{Applied}, \mathsf{Rejected}\}$;

$\mathsf{fwVer}, \mathsf{lsv}, \mathsf{capVer}, \mathsf{digest} \in \mathbb{N}$ are, respectively, the running firmware version, the stored lowest-supported-version floor, the staged capsule version, and an abstract digest of the staged image; and $\mathsf{capPresent}, \mathsf{sigOK}, \mathsf{reset} \in \mathbb{B}$ record whether a capsule is staged, whether its digest authenticated, and whether a reset has occurred since staging.

Definition 2 (Initial states)**.** $\textsc{Init}(s) \triangleq \mathsf{phase}(s) = \mathsf{Idle} \wedge \neg\mathsf{capPresent}(s) \wedge \neg\mathsf{reset}(s) \wedge \mathsf{lsv}(s) \leq \mathsf{fwVer}(s)$.

The transition relation $\rightarrowtriangle \subseteq \mathcal{S} \times \mathcal{S}$ is defined by the rules in Fig. 2. We write $s \rightarrowtriangle s'$ for a single step. The staged digest is chosen in Stage, while BeginAuth computes the signature verdict $\mathsf{sigOK} := \mathsf{certOK}(\mathsf{digest})$ from it. The Apply and Reject guards are *complementary*: from Auth, exactly one of them is enabled, so the model has no stuck non-sink states.

Definition 3 (Run)**.** A *run* is a function $\sigma : \mathbb{N} \to \mathcal{S}$ that starts initial and is closed under $\rightarrowtriangle$:

$$\mathrm{IsRun}(\sigma) \triangleq \textsc{Init}(\sigma\,0) \wedge \forall i \in \mathbb{N}.\ \sigma\,i \rightarrowtriangle \sigma\,(i{+}1).$$

The Stutter and Finish rules guarantee that every reachable state has a successor, so IsRun is non-vacuous: every finite prefix extends to an infinite run.

a) Threat model.: The fields capVer and digest are adversary-controlled *inputs*: Stage lets the environment pick any version v and any abstract digest d, modelling an attacker who submits arbitrary (including forged or downgraded) capsules. Signature validity is *not* fixed at Stage: BeginAuth computes $\mathsf{sigOK} := \mathsf{certOK}(\mathsf{digest})$, so application depends on the staged image contents rather than an unconstrained bit. Here $\mathsf{certOK}(\mathsf{digest})$ is an abstract acceptance predicate for the complete authenticated capsule: it subsumes the signature, certificate chain, trust anchors, and the binding of the version metadata to the hashed bytes. It is not a claim that signature validity is a property of a digest alone. The checker remains a trusted oracle—we verify the update *decision logic*, not the cryptographic primitive. Our guarantees are thus conditional on the soundness of the underlying signature verification, exactly the decomposition used by Ironclad [13] and Komodo [14].

IV. An LTL Embedding in Lean 4

We embed LTL *shallowly*: a temporal proposition is a predicate on a run and a position, $\mathsf{TProp} \triangleq (\mathbb{N} \to \mathcal{S}) \to \mathbb{N} \to \mathrm{Prop}$. Table I gives the operators and their semantics; each is a one-line Lean definition.

Listing 1 shows the embedding verbatim. Because the embedding is shallow, duality laws such as $\mathbf{F}\,\varphi \equiv \neg\,\mathbf{G}\,\neg\varphi$ are theorems of the meta-logic rather than primitives, and we never need a separate LTL proof system: an LTL validity is just a Lean proposition, discharged by ordinary tactics.

TABLE I
Shallow LTL semantics. $[\![\varphi]\!](\sigma, i)$ is the truth of φ on run σ at position i.

Operator	Semantics
$\mathsf{now}\,p$	$[\![\mathsf{now}\,p]\!](\sigma,i) = p(\sigma\,i)$
$\mathbf{X}\,\varphi$	$[\![\mathbf{X}\,\varphi]\!](\sigma,i) = [\![\varphi]\!](\sigma,i{+}1)$
$\mathbf{G}\,\varphi$	$[\![\mathbf{G}\,\varphi]\!](\sigma,i) = \forall j \geq i.\ [\![\varphi]\!](\sigma,j)$
$\mathbf{F}\,\varphi$	$[\![\mathbf{F}\,\varphi]\!](\sigma,i) = \exists j \geq i.\ [\![\varphi]\!](\sigma,j)$
$\varphi \Rightarrow \psi$	$[\![\varphi]\!](\sigma,i) \to [\![\psi]\!](\sigma,i)$
$\vDash \varphi$ (valid)	$\forall\sigma.\ \mathrm{IsRun}(\sigma) \to [\![\varphi]\!](\sigma,0)$

V. Properties and Proofs

We now state the five security properties and their machine-checked proofs. All five are validities ($\vDash$) over the model of Section III. The proof architecture (Fig. 3) is layered: transition-level lemmas about a single $\rightarrowtriangle$ feed inductive invariants over runs, which in turn discharge the $\mathbf{G}$ properties; the liveness property is proved by an explicit three-step witness.

A. Transition-level lemmas

Lemma 1 (LSV monotonicity per step)**.** *For all s, s', if $s \rightarrowtriangle s'$ then $\mathsf{lsv}(s) \leq \mathsf{lsv}(s')$.*

Proof. By case analysis on the rule deriving $s \rightarrowtriangle s'$. Every rule except Apply leaves lsv unchanged, giving $\mathsf{lsv}(s) = \mathsf{lsv}(s')$. The Apply rule requires $\mathsf{lsv}(s) \leq \mathsf{capVer}(s)$ and sets $\mathsf{lsv}(s')$=$\mathsf{capVer}(s)$, hence $\mathsf{lsv}(s) \leq \mathsf{lsv}(s')$. In Lean this is the lemma `lsv_monotone`. □

Lemma 2 (Apply guard)**.** *If $s \rightarrowtriangle s'$, $\mathsf{phase}(s') = \mathit{Applied}$, and $\mathsf{phase}(s) \neq \mathit{Applied}$, then $\mathsf{sigOK}(s)$ and $\mathsf{lsv}(s) \leq \mathsf{capVer}(s)$.*

Proof. The only rule that produces a successor in phase Applied from a non-Applied state is Apply, whose premises are exactly $\mathsf{sigOK}(s)$ and $\mathsf{lsv}(s) \leq \mathsf{capVer}(s)$. All other rules are eliminated by `simp_all`. □

Lemma 3 (Forced good path)**.** (i) *If $\mathsf{phase}(s) = \mathit{Staged}$ and $s \rightarrowtriangle s'$ then $\mathsf{phase}(s') = \mathit{PostReset}$ and s' preserves $\mathsf{fwVer}, \mathsf{capVer}, \mathsf{lsv}$, and digest.* (ii) *If $\mathsf{phase}(s) = \mathit{PostReset}$ and $s \rightarrowtriangle s'$ then $\mathsf{phase}(s') = \mathit{Auth}$, s' preserves $\mathsf{fwVer}, \mathsf{capVer}$, and lsv, and $\mathsf{sigOK}(s') = \mathsf{certOK}(\mathsf{digest}(s))$.* (iii) *If $\mathsf{phase}(s) = \mathit{Auth}$, $\mathsf{sigOK}(s)$, $\mathsf{lsv}(s) \leq \mathsf{capVer}(s)$, and $s \rightarrowtriangle s'$, then $\mathsf{phase}(s') = \mathit{Applied}$.*

Proof. Each part is a case analysis in which all but one rule is disabled by the phase premise. For *(iii)* the Reject rule is additionally disabled because its guard $\neg\mathsf{sigOK}(s) \vee \mathsf{capVer}(s) < \mathsf{lsv}(s)$ contradicts the hypotheses ($\mathsf{sigOK}(s)$ and $\mathsf{lsv}(s) \leq \mathsf{capVer}(s)$); `omega` discharges the arithmetic contradiction. Hence Apply is the unique enabled rule. □

$$\frac{\mathsf{phase}(s) = \mathsf{Idle}}{s \twoheadrightarrow s}\ (\textsc{Stutter}) \qquad \frac{\mathsf{phase}(s) = \mathsf{Staged}}{s \twoheadrightarrow s[\,\mathsf{phase}{:=}\mathsf{PostReset},\ \mathsf{reset}{:=}\top\,]}\ (\textsc{Reset}) \qquad \frac{\mathsf{phase}(s) = \mathsf{PostReset}}{s \twoheadrightarrow s[\,\mathsf{phase}{:=}\mathsf{Auth},\ \mathsf{sigOK}{:=}\mathsf{certOK}(\mathsf{digest}(s))\,]}\ (\textsc{BeginAuth})$$

$$\frac{\mathsf{phase}(s) = \mathsf{Idle}}{s \twoheadrightarrow s[\,\mathsf{phase}{:=}\mathsf{Staged},\ \mathsf{capVer}{:=}v,\ \mathsf{digest}{:=}d,\ \mathsf{capPresent}{:=}\top\,]}\ (\textsc{Stage}) \qquad \frac{\mathsf{phase}(s) \in \{\mathsf{Applied}, \mathsf{Rejected}\}}{s \twoheadrightarrow s[\,\mathsf{phase}{:=}\mathsf{Idle},\ \mathsf{capPresent}{:=}\bot,\ \mathsf{reset}{:=}\bot\,]}\ (\textsc{Finish})$$

$$\frac{\mathsf{phase}(s) = \mathsf{Auth} \quad \mathsf{sigOK}(s) \quad \mathsf{lsv}(s) \le \mathsf{capVer}(s)}{s \twoheadrightarrow s[\,\mathsf{phase}{:=}\mathsf{Applied},\ \mathsf{fwVer}{:=}\mathsf{capVer}(s),\ \mathsf{lsv}{:=}\mathsf{capVer}(s)\,]}\ (\textsc{Apply}) \qquad \frac{\mathsf{phase}(s) = \mathsf{Auth} \quad \neg\mathsf{sigOK}(s) \ \vee\ \mathsf{capVer}(s) < \mathsf{lsv}(s)}{s \twoheadrightarrow s[\,\mathsf{phase}{:=}\mathsf{Rejected},\ \mathsf{capPresent}{:=}\bot\,]}\ (\textsc{Reject})$$

Fig. 2. Inference rules of the transition relation $\twoheadrightarrow$. Stage chooses a staged version and digest; BeginAuth computes sigOK := certOK(digest); and Apply advances both fwVer and lsv to capVer. The Apply and Reject guards are mutually exclusive and exhaustive, so from Auth exactly one fires.

```
abbrev TProp := (Nat → St) → Nat → Prop

/-- Lift a state predicate to "holds now". -/
def now (p : St → Prop) : TProp := fun σ i => p (σ i)

/-- G φ : φ holds at every position from here on. -/
def G (P : TProp) : TProp := fun σ i => ∀ j, i ≤ j → P σ
    j

/-- F φ : φ holds at some position from here on. -/
def F (P : TProp) : TProp := fun σ i => ∃ j, i ≤ j ∧ P σ
    j

/-- X φ : φ holds at the next position. -/
def X (P : TProp) : TProp := fun σ i => P σ (i + 1)

def Imp (P Q : TProp) : TProp := fun σ i => P σ i → Q σ i
def And (P Q : TProp) : TProp := fun σ i => P σ i ∧ Q σ i

/-- A temporal property is *valid* if it holds at position 0
    of every run. -/
def Sat (φ : TProp) : Prop := ∀ σ, IsRun σ → φ σ 0
```

Listing 1. The LTL embedding, extracted verbatim from the kernel-checked Lean 4 source.



Fig. 3. Proof architecture. Single-step facts are lifted to run invariants by induction on position, then to **G** validities; liveness reuses the "forced" step lemmas without any fairness hypothesis.

Lemma 3(iii) is the crux of the liveness argument: in the good case the system is *deterministic up to the sink*, so no fairness constraint is needed to guarantee progress.

B. Inductive invariants

Definition 4 (Invariants)**.**

$$\begin{aligned}\mathrm{InvSig}(s) &\triangleq \mathsf{phase}(s) = \mathsf{Applied} \to \mathsf{sigOK}(s),\\ \mathrm{InvReset}(s) &\triangleq \mathsf{phase}(s) \in \{\mathsf{PostReset}, \mathsf{Auth}, \mathsf{Applied}\}\\ &\qquad \to \mathsf{reset}(s),\\ \mathrm{InvFloor}(s) &\triangleq \mathsf{lsv}(s) \le \mathsf{fwVer}(s).\end{aligned}$$

Lemma 4 (Invariants hold on all runs)**.** *If* $\mathrm{IsRun}(\sigma)$ *then* $\forall i.\ \mathrm{InvSig}(\sigma\, i)$, $\forall i.\ \mathrm{InvReset}(\sigma\, i)$, *and* $\forall i.\ \mathrm{InvFloor}(\sigma\, i)$.

```
theorem invReset_step {s s'} (h : Step s s') (hs : InvReset
    s) : InvReset s' := by
  intro hmem
  cases h with
  | beginAuth hp => exact hs (Or.inl hp)
  | apply hp _ _ => exact hs (Or.inr (Or.inl hp))
  | _ => simp_all
```

Listing 2. Inductive step for the reset-ordering invariant (extracted from source).

Proof. Each is proved by induction on i. *Base:* $\textsc{Init}(\sigma\,0)$ forces $\mathsf{phase}(\sigma\,0) = \mathsf{Idle}$ and $\mathsf{lsv}(\sigma\,0) \le \mathsf{fwVer}(\sigma\,0)$, so the three invariants hold. *Step:* assume the invariant at i and analyse the rule for $\sigma\, i \twoheadrightarrow \sigma\,(i{+}1)$. For InvSig, only Apply reaches Applied, and it requires sigOK. For InvReset, Reset sets reset before any of $\{\mathsf{PostReset}, \mathsf{Auth}, \mathsf{Applied}\}$ is entered, and the rules BeginAuth/Apply that move *within* this set inherit reset from the predecessor by the induction hypothesis. InvReset must be stated over the *set* $\{\mathsf{PostReset}, \mathsf{Auth}, \mathsf{Applied}\}$ rather than Applied alone; the singleton form is not inductive. Finally, InvFloor is preserved because Apply sets both fwVer and lsv to capVer, while every other rule leaves those fields unchanged. □

Listing 2 shows the InvReset inductive step, including the explicit handling of the two phase-preserving rules that a naive `simp_all` cannot close.

C. The temporal theorems

Property 1 (Authenticity — safety)**.**

$$\vDash\ \mathbf{G}\,\big(\mathsf{now}\,(\mathsf{phase}{=}\mathsf{Applied})\ \Rightarrow\ \mathsf{now}\,\mathsf{sigOK}\big).$$

Whenever the firmware reports a capsule as applied, that capsule's signature was valid: an unauthenticated image is never applied.

Proof. Unfold $\vDash$ and **G**: fix a run σ and a position j; the goal is $\mathrm{InvSig}(\sigma\, j)$, which is Lemma 4. □

Property 2 (Anti-rollback — local)**.** $\vDash\ \mathbf{G}\,\big(\mathsf{now}\,\mathsf{lsv} \le \mathbf{X}\,(\mathsf{now}\,\mathsf{lsv})\big)$. The stored lowest-supported-version floor never decreases across a single step.

Proof. Immediate from Lemma 1 applied to $\sigma\, j \twoheadrightarrow \sigma\,(j{+}1)$. □

Property 3 (Anti-rollback — global)**.** For every run σ and all positions $i \le j$, $\mathsf{lsv}(\sigma\, i) \le \mathsf{lsv}(\sigma\, j)$. No sequence of capsule operations can ever roll the stored anti-rollback floor backward.

Proof. By induction on the derivation of $i \le j$. The reflexive case is $\mathsf{lsv}(\sigma\, i) \le \mathsf{lsv}(\sigma\, i)$; the successor case composes the induction hypothesis with Lemma 1 by transitivity of $\le$ on $\mathbb{N}$. □

Because anti-rollback is now tracked by the stored floor rather than the running firmware version, we also maintain the inductive side condition $\mathrm{InvFloor}(s) \triangleq \mathsf{lsv}(s) \le \mathsf{fwVer}(s)$ on every reachable state. This records that the floor-based EDK II check never outruns the version the platform is currently booting.

Property 4 (Apply guard)**.**

$$\vDash\ \mathbf{G}\Big(\big(\mathsf{now}\,(\mathsf{phase}{\neq}\mathsf{Applied}) \wedge \mathbf{X}\ \mathsf{now}\,(\mathsf{phase}{=}\mathsf{Applied})\big) \Rightarrow\ \mathsf{now}\,(\mathsf{sigOK} \wedge \mathsf{lsv} \le \mathsf{capVer})\Big).$$

Every *entry* into the applied state is guarded by a valid signature and a capsule version at or above the LSV floor.

Proof. Directly from Lemma 2 on the step $\sigma\, j \twoheadrightarrow \sigma\,(j{+}1)$. Note the conclusion is evaluated at position j (the pre-state): see Section VII. □

Property 5 (Reset ordering)**.** $\vDash \mathbf{G}\,\big(\mathsf{now}\,(\mathsf{phase}{=}\mathsf{Applied}) \Rightarrow \mathsf{now}\,\mathsf{reset}\big)$. A capsule is applied only after the system has reset.

Proof. Specialize the InvReset instance from Lemma 4 (the Applied disjunct). □

Property 6 (Responsiveness — liveness)**.** Let $\mathsf{good}(s) \triangleq \mathsf{phase}(s){=}\mathsf{Staged} \wedge \mathsf{certOK}(\mathsf{digest}(s)) \wedge \mathsf{lsv}(s) \le \mathsf{capVer}(s)$. Then

$$\vDash\ \mathbf{G}\,\big(\mathsf{now}\,\mathsf{good}\ \Rightarrow\ \mathbf{F}\ \mathsf{now}\,(\mathsf{phase}{=}\mathsf{Applied})\big).$$

In the interruption-free model, once a staged capsule's digest authenticates and its version is at or above the stored floor, the run eventually reaches the applied state.

Proof. Fix a run σ and a position j with $\mathsf{good}(\sigma\, j)$. By Lemma 3(i), $\mathsf{phase}(\sigma\,(j{+}1)) = \mathsf{PostReset}$ and $\mathsf{fwVer}, \mathsf{capVer}, \mathsf{lsv}$, and digest are preserved; by (ii), $\mathsf{phase}(\sigma\,(j{+}2)) = \mathsf{Auth}$ with $\mathsf{sigOK}(\sigma\,(j{+}2)) = \mathsf{certOK}(\mathsf{digest}(\sigma\, j))$; the preserved digest and floor then satisfy the hypotheses of (iii), giving $\mathsf{phase}(\sigma\,(j{+}3)) = \mathsf{Applied}$. The witness $j{+}3 \ge j$ discharges $\mathbf{F}$. □

Remark 1 (No fairness needed). Property 6 requires *no* fairness assumption. The standard liveness recipe assumes weak/strong fairness to rule out a scheduler that starves an enabled action [17], [16]. Here Lemma 3 shows the good path is the *only* enabled path inside the non-Idle phases—Stutter is disabled outside Idle, and Reject is disabled by the guard—so progress is forced by determinism. This is a property of the interruption-free abstraction, not of arbitrary firmware executions: an external interruption such as power failure between reset, authentication, and `SetImage` would falsify the liveness claim. Within the model's forced good path, however, the sole continuation is indeed the apply step.

```
theorem responsiveness : Sat (G (Imp goodStaged (F
    appliedNow))) := by
  intro σ hr j _ hgood
  obtain ⟨hStaged, hSig, hFloor⟩ := hgood
  -- step 1: reset  (CapsuleStaged → PostReset), version/
     digest fields preserved
  obtain ⟨h1p, h1fw, h1ver, h1lsv, h1digest⟩ :=
     reset_forced hStaged (hr.2 j)
  -- step 2: beginAuth (PostReset → Authenticating), sig
     computed from digest
  obtain ⟨h2p, h2fw, h2ver, h2lsv, h2sig⟩ := auth_forced
     h1p (hr.2 (j + 1))
  -- step 3: apply forced (authentic + at-or-above-floor ⇒
      Applied)
  have h3 : (σ (j + 3)).phase = Applied := by
    apply good_apply_forced h2p
    · rw [h2sig, h1digest]; exact hSig
    · rw [h2lsv, h1lsv, h2ver, h1ver]; exact hFloor
    · exact hr.2 (j + 2)
  exact ⟨j + 3, by omega, h3⟩
```

Listing 3. Fairness-free responsiveness, extracted verbatim from the kernel-checked Lean 4 source.

TABLE II
THE MACHINE-CHECKED GUARANTEES DISCUSSED IN THE PAPER. ALL ARE KERNEL-VERIFIED; NONE USES `SORRY`.

ID	Class	Statement (informal)
S	safety	$\mathbf{G}(\mathsf{Applied} \to \mathsf{sigOK})$ — only authentic images applied
R1	safety	$\mathbf{G}(\mathsf{lsv} \le \mathbf{X}\,\mathsf{lsv})$ — per-step floor monotone
R2	safety	$i \le j \Rightarrow \mathsf{lsv}_i \le \mathsf{lsv}_j$ — global floor monotone
F	safety	$\mathbf{G}(\mathsf{lsv} \le \mathsf{fwVer})$ — floor never exceeds running version
G	safety	entry to Applied is signature- and floor-guarded
O	safety	$\mathbf{G}(\mathsf{Applied} \to \mathsf{reset})$ — reset precedes apply
L	liveness	$\mathbf{G}(\mathsf{good} \to \mathbf{F}\ \mathsf{Applied})$ — forced good-path progress

Listing 3 reproduces the Lean proof of Property 6 verbatim, showing how the three forced steps chain and how the field-preservation equalities are rewritten to satisfy the apply guard.

The verified guarantees are summarized in Table II.

```
'safety_authentic'     depends on axioms: [propext]
'antirollback_lsv'       depends on axioms: [propext, Quot.
    sound]
'antirollback_lsv_global' depends on axioms: [propext, Quot
    .sound]
'lsv_floor_maintained'   depends on axioms: [propext]
'apply_guarded'          depends on axioms: [propext]
'reset_first'            depends on axioms: [propext]
'responsiveness'         depends on axioms: [propext, Quot.
    sound]
```

Listing 4. Kernel axiom audit. `sorryAx` is absent from every closure; only the two standard logical axioms appear.

VI. Trust Base and Axiom Audit

A machine-checked proof is only as trustworthy as its kernel and its axioms. Lean 4 [20] is founded on a dependent type theory (the Calculus of Inductive Constructions [21]) with a small trusted kernel; the elaborator and tactics are *untrusted*, since every proof term they produce is independently re-checked by the kernel. We therefore make two trust claims and substantiate both mechanically.

First, *no proof uses an escape hatch.* A textual scan finds no `sorry`, `admit`, `axiom`, `native_decide`, or `unsafe` in the development. Second, and more authoritatively, we query the kernel itself with `#print axioms` on each theorem; the closure of axiom dependencies is:

$$\{\,\texttt{propext},\ \texttt{Quot.sound}\,\}.$$

These are the two standard, sound axioms of Lean's logic (propositional extensionality and the soundness of quotients); `Classical.choice` is not needed, and—decisively—the special marker `sorryAx` that any `sorry` would introduce is *absent*. The full audit output is shown in Listing 4.

The trusted computing base of the *argument* is thus: the Lean kernel, the two axioms above, and—crucially—the *fidelity of the model* to the real firmware. The first two are shared with large verified artifacts such as Mathlib [22]; the third is the subject of Section VII and is the honest limit of any such result, exactly as in seL4 [11] and CompCert [12].

VII. From Model to EDK II

Our corrected model abstracts the EDK II `FmpDevicePkg` control flow. The phase correspondence is direct: Stage is `UpdateCapsule` populating the capsule region with a candidate version and digest; Reset is the platform reset with capsule persistence; BeginAuth is post-reset coalescing plus FMP discovery and signature verification; and Apply/Reject are the two outcomes of `CheckImage` followed by `SetImage`. Making lsv explicit matches the effective lowest-supported-version value used by the shipped check, and deriving sigOK at authentication time matches the fact that acceptance is computed after staging rather than chosen by the environment.

Algorithm 1: Abstracted FMP `CheckImage`: shipped LSV check and an abstract capsule-authentication decision.

input : capsule image I; effective lowest-supported version lsv
output : accept (apply) or reject

$(\mathsf{capVer}, \mathrm{cert}) \leftarrow \textsc{ParseHeader}(I)$
$\mathsf{digest} \leftarrow \textsc{Hash}(I)$
if $\neg$ *VerifySignature*(digest, *cert*) **then**
  return reject // ¬sigOK
if capVer < lsv **then**
  return reject // EDK II anti-rollback check
return accept // sigOK ∧ lsv ≤ capVer

a) A temporal ambiguity the proof surfaced.: Our first attempt stated the apply guard (Property 4) on the *post*-state: "after entering Applied, the installed version is strictly above the anti-rollback floor." Lean rejected it. The reason is structural: Apply executes fwVer:=capVer and, in this model, lsv:=capVer, so in the post-state the relevant relation is an *equality*, not a strict inequality. The guarantee is necessarily a property of the *pre*-application state. This is not a curiosity: implementations and audits that phrase anti-rollback as a post-condition on the installed image are conflating two distinct states, and a check written against the post-state would be tautological.

b) The LSV fidelity boundary.: The blind-review version of the model used the stronger guard fwVer < capVer, which proved monotonicity of the running firmware version but did not match the reference implementation. Shipped EDK II checks the incoming version against the effective lowest-supported version lsv, not directly against the currently running version fwVer. In `FmpDxe`, that effective value is the maximum of the build-time floor, a device-reported floor, and a monotone UEFI-variable floor. After a successful `SetImage`, EDK II reads a lowest-supported-version value from the authenticated payload header and raises the variable floor only when that value is larger.

Our Apply rule intentionally specializes this mechanism to the common policy lsv:=capVer: the newly installed image becomes the next minimum acceptable version. Consequently, the proved monotonicity and lsv ≤ fwVer invariant apply directly to this policy instance, while the pre-state guard lsv ≤ capVer corresponds to the shipped `CheckImage` comparison. A bit-for-bit model of `FmpDxe` would add the payload-declared floor as a separate authenticated field and update lsv:= max(lsv, payloadLSV); the present paper does not claim that the assignment lsv:=capVer is universal EDK II behavior.

c) Excluded attack classes and model assumptions.: The model deliberately abstracts several operational attack vectors, separating explicit assumptions from future

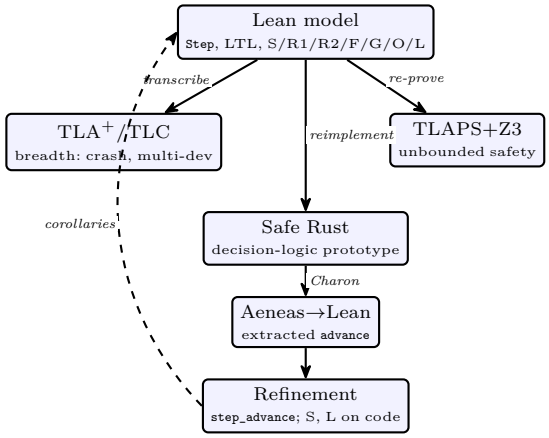


Fig. 4. The artifact chain. One specification is cross-validated by TLC (breadth), re-proved unbounded by TLAPS, and reimplemented in Rust; Aeneas extracts the Rust to Lean, where it is shown to refine the proved `Step` relation under the stated abstraction assumptions, so the abstract theorems apply to the prototype code.

verification work:

- *TOCTOU and image integrity.* The corrected model stores an abstract digest at Stage and derives sigOK from it in BeginAuth. It still assumes that the bytes hashed into digest are the bytes later authenticated and applied; modelling mutable staging memory, partial writes, or concrete cryptographic digests is future work.
- *DMA and SMM concurrency.* Concurrent memory writes by DMA-capable peripherals or System Management Mode (SMM) preemption during evaluation are unrepresented; we assume staging memory is isolated prior to BeginAuth.
- *Power-fail interruptions.* Unscheduled resets during Apply falsify the responsiveness claim, isolating crash consistency of version records as an independent obligation.
- *Multiple FMP devices* and dependency expressions (`EFI_FIRMWARE_IMAGE_DEP`) are not modelled; the version field would become a per-device vector with a partial order.

None of these changes the shape of the proofs; each enlarges the state record and adds rules, with the invariant-and-forced-step methodology carried over.

VIII. From Specification to Executable Code

The proofs of Section V establish the paper's guarantees for an *abstract* model, and Section VI is candid that the residual assumption is the fidelity of that model to firmware. We separately reduce the specification-to-code gap by carrying the same decision logic across four artifacts (Fig. 4): the Lean model checked here, an independent explicit-state cross-check, an unbounded re-proof, and an executable safe-Rust prototype related back to the Lean `Step` relation. This chain validates the prototype, not the EDK II implementation.

a) Breadth and unbounded safety.: Transcribing the model action-for-action into TLA$^+$ [23] and running TLC reproduces the headline guarantees on the shared abstraction (~157 reachable states), and—perturbing the model along axes the Lean version abstracts away—shows that fairness-free responsiveness (L) is an artifact of atomicity (a power-fail action falsifies it while S, R1, G, O survive) and that scalar anti-rollback does not lift to multiple FMP devices with `EFI_FIRMWARE_IMAGE_DEP` dependencies. The bounded version range TLC requires is then lifted: TLAPS [24] discharges the same inductive invariant (TypeOK $\wedge$ InvSig $\wedge$ InvReset) and the per-step anti-rollback fact over all of $\mathbb{N}$ via Z3, so the safety core is machine-checked twice by tools with disjoint trusted bases.

b) Refinement to Rust.: We reimplement the good-path decision logic as a deterministic, safe-Rust function `advance` and extract it to Lean with the Charon/Aeneas toolchain [25]. The extracted (monadic, `u64`-typed) function is *proved* to refine a Lean definition `advance` over the model's state, via an abstraction map α, on all non-sink phases (`CapsuleBridge.advance_refines`, kernel-checked, `sorryAx` absent). The bridge is conditional on an explicit `VerifierRefines` hypothesis: the concrete verifier result must equal the abstract certOK oracle. This is the cryptographic refinement boundary, not a proof of the verifier itself. For the abstract `advance` we prove (axiom base {`propext`, `Quot.sound`}, no `sorry`):

Lemma 5 (Refinement)**.** $\forall s.\ s \twoheadrightarrow \mathsf{advance}(s)$*: every* `advance` *step is a legal transition of the proved system.*

Iterating the abstract `advance` from an initial state therefore yields a legal `IsRun` (`detRun_isRun`), and the abstract theorems transfer to the decision-logic prototype *as corollaries*, with no new liveness or safety proof:

Corollary 1 (Code-level guarantees)**.** *On the run corresponding to the Rust* `advance`*:* (S) *an applied capsule was signature-valid (*`safety_authentic_det`*), and* (L) *a good staged capsule is eventually applied (*`responsiveness_det`*)—the latter the executable image of the forced three-step witness.*

This is the refinement the conventional decomposition demands: the cryptographic primitive and hardware I/O remain a trusted oracle (as for Ironclad [13] and Komodo [14]), while the update *decision logic*—the part the proofs constrain—is now executable prototype code whose behaviour is related to the temporal specification. This does not verify EDK II control flow, PKCS#7 verification, capsule parsing, or flash I/O. Liveness under adversarial interruption is the one guarantee that does not cross: it is false without a fairness assumption and stays in the TLA$^+$ layer.

IX. Related Work

a) Verified systems software.: seL4 [11] proved functional correctness and security of an OS kernel; Com-

pCert [12] a verified C compiler; Ironclad [13] and the Komodo enclave monitor [14] end-to-end and enclave guarantees; and Serval [26] scaled automated verification to systems code. We target a narrower but under-examined artifact—the *firmware update* state machine—and contribute the first temporal-logic treatment of it.

b) Temporal logic and model checking.: LTL [15] and its safety/liveness taxonomy [18], [19] underpin model checkers such as SPIN [27] and the TLA$^+$ framework [23]. Model checking would let us *discover* counterexamples automatically but bounds the state space; our deductive, parameter-polymorphic proof (versions range over all of $\mathbb{N}$) gives an *unbounded* guarantee, at the cost of manual proof. The two are complementary: a TLA$^+$ or SPIN model is a natural front-end for triaging designs before mechanization.

c) Firmware security.: The attack literature [1], [2], [3], [4], [5] and the NIST guidelines [6], [7] establish the importance of authenticated, anti-rollback firmware update, but treat the update logic informally. Yao and Zimmer [28] provide a comprehensive treatment of secure firmware principles—including capsule update and recovery—but without machine-checked proofs. The UEFI standard itself is surveyed in depth in [29]. EDK II [10] and `FmpDevicePkg` [9] are the reference implementation we abstract. To our knowledge, no prior work gives a machine-checked temporal-logic specification and proof of the UEFI capsule update flow.

X. Conclusion

We have given a compact, machine-checked temporal-logic account of the UEFI capsule update process. From a compact interruption-free labelled-transition model and a shallow LTL embedding in Lean 4, we proved authenticity, floor-based anti-rollback, the apply guard, reset ordering, and a qualified responsiveness result that holds only along the model's forced good path. We also audited the kernel axiom base to confirm that the proofs rest only on the two standard logical axioms with no `sorry`. Beyond the guarantees themselves, the exercise demonstrates a methodology—transition lemmas, inductive invariants, and forced-path liveness—that scales to the richer FMP feature set and, by composing with a verified signature checker, toward a fully verified firmware-update reference monitor. The key lesson is not that the model captures every firmware contingency, but that mechanization forces the security-relevant assumptions to become explicit.

a) Availability.: The complete artifact suite—the Lean 4 proof development, TLA$^+$ specifications with TLC configurations and TLAPS proofs, and the Aeneas/Rust refinement pipeline—is available open-source at https://github.com/vincentjzimmer/formal/tree/main/capsule-ltl.

References

[1] T. Hudson and L. Rudolph, "Thunderstrike: EFI firmware bootkits for Apple MacBooks," in *Proc. 8th ACM Int. Systems and Storage Conf. (SYSTOR)*, 2015.

[2] ESET Research, "LoJax: First UEFI rootkit found in the wild, courtesy of the Sednit group," ESET White Paper, 2018.

[3] ——, "BlackLotus UEFI bootkit: Myth confirmed," https://www.welivesecurity.com/2023/03/01/blacklotus-uefi-bootkit-myth-confirmed/, 2023.

[4] Eclypsium, "There's a hole in the boot," CVE-2020-10713, 2020.

[5] Quarkslab, "PixieFail: Nine vulnerabilities in Tianocore EDK II's network stack," Quarkslab blog; CERT/CC `VU#132380`, 2024. [Online]. Available: https://kb.cert.org/vuls/id/132380

[6] A. Regenscheid, "BIOS protection guidelines for servers," *NIST Special Publication 800-147B*, 2014.

[7] ——, "Platform firmware resiliency guidelines," *NIST Special Publication 800-193*, 2018.

[8] UEFI Forum, "Unified extensible firmware interface (uefi) specification, version 2.10," https://uefi.org/specifications, 2022, see §8.5.3 (`UpdateCapsule`) and §23 (Firmware Update and Reporting).

[9] TianoCore Community, "EDK II FmpDevicePkg and the firmware management protocol," https://github.com/tianocore/edk2/tree/master/FmpDevicePkg, 2024, implements `CheckImage`/`SetImage` with lowest-supported-version anti-rollback.

[10] ——, "Edk ii: The open-source UEFI reference implementation," https://github.com/tianocore/edk2, 2024.

[11] G. Klein, K. Elphinstone, G. Heiser *et al.*, "seL4: Formal verification of an OS kernel," in *Proc. 22nd ACM Symp. on Operating Systems Principles (SOSP)*, 2009, pp. 207–220.

[12] X. Leroy, "Formal verification of a realistic compiler," *Communications of the ACM*, vol. 52, no. 7, pp. 107–115, 2009.

[13] C. Hawblitzel, J. Howell, J. R. Lorch *et al.*, "Ironclad apps: End-to-end security via automated full-system verification," in *Proc. 11th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2014, pp. 165–181.

[14] A. Ferraiuolo, A. Baumann, C. Hawblitzel, and B. Parno, "Komodo: Using verification to disentangle secure-enclave hardware from software," in *Proc. 26th ACM Symp. on Operating Systems Principles (SOSP)*, 2017, pp. 287–305.

[15] A. Pnueli, "The temporal logic of programs," in *Proc. 18th IEEE Symp. on Foundations of Computer Science (FOCS)*, 1977.

[16] Z. Manna and A. Pnueli, *Temporal Verification of Reactive Systems: Safety*. Springer, 1995.

[17] C. Baier and J.-P. Katoen, *Principles of Model Checking*. MIT Press, 2008.

[18] B. Alpern and F. B. Schneider, "Defining liveness," *Information Processing Letters*, vol. 21, no. 4, pp. 181–185, 1985.

[19] L. Lamport, "Proving the correctness of multiprocess programs," *IEEE Trans. Softw. Eng.*, vol. SE-3, no. 2, pp. 125–143, 1977.

[20] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Proc. 28th Int. Conf. on Automated Deduction (CADE)*, ser. LNCS, vol. 12699. Springer, 2021, pp. 625–635.

[21] T. Coquand and G. Huet, "The calculus of constructions," *Information and Computation*, vol. 76, no. 2–3, pp. 95–120, 1988.

[22] M. Community, "The Lean mathematical library," in *Proc. 9th ACM SIGPLAN Int. Conf. on Certified Programs and Proofs (CPP)*. ACM, 2020.

[23] L. Lamport, "The temporal logic of actions," *ACM Trans. Program. Lang. Syst.*, vol. 16, no. 3, pp. 872–923, 1994.

[24] D. Cousineau, D. Doligez, L. Lamport, S. Merz, D. Ricketts, and H. Vanzetto, "TLA+ proofs," in *Proc. 18th Int. Symp. on Formal Methods (FM)*, ser. LNCS, vol. 7436, 2012, pp. 147–154.

[25] S. Ho and J. Protzenko, "Aeneas: Rust verification by functional translation," *Proc. ACM Program. Lang. (ICFP)*, vol. 6, pp. 711–741, 2022.

[26] L. Nelson, J. Bornholt, R. Gu *et al.*, "Scaling symbolic evaluation for automated verification of systems code with Serval," in *Proc. 27th ACM Symp. on Operating Systems Principles (SOSP)*, 2019, pp. 225–242.

[27] G. J. Holzmann, "The model checker SPIN," *IEEE Trans. Softw. Eng.*, vol. 23, no. 5, pp. 279–295, 1997.

[28] J. Yao and V. Zimmer, *Building Secure Firmware: Armoring the Foundation of the Platform*. Apress, 2020.

[29] V. Zimmer, S. Marisetty, and M. Rothman, *Beyond BIOS: Developing with the Unified Extensible Firmware Interface, Third Edition*. De Gruyter, 2017.